PDP gallery, image derivatives, and LazyImage

Audience: Engineers working on the storefront, catalog API, admin uploads, or image sync.

Last updated: 2026-09-16

Related:

- SKU-wise print mapping & per-print galleries
- Swatch URL fallback
- Product images & Ledgerly sync
- Frontend architecture

TL;DR

Layer	Responsibility
DB / API	`images_json`, `color_images_json`, `colors_json` (per-print galleries + `swatchUrl`)
`galleryForSelectedColor()`	Resolves hero + gallery URLs for selected print/size/SKU
`storefront-image.ts`	Maps catalog URLs → `.thumb.jpg` (240px) / `.display.jpg` (800px) / original (1250px)
`LazyImage`	Shared `<img>` wrapper: native lazy load, `fetchPriority`, fallback URL, in-memory warm cache
`ProductGallery`	PDP hero, side rail, blur placeholder, prefetch neighbours + idle warmup
`VariantPickers`	Swatch thumbs + hover prefetch before colour change

The PDP loads the **hero first** at display resolution; side thumbnails and off-screen images stay lazy or low priority so they do not block LCP.

Table of contents

1. End-to-end data flow
2. Gallery resolution (`galleryForSelectedColor`)
3. Image derivatives (`storefront-image`)

4. LazyImage component
5. ProductGallery UI
6. Colour swatches & prefetch on intent
7. PDP route wiring
8. Backend & admin
9. Performance strategy
10. Testing
11. Troubleshooting
12. Future work
13. Code index

---

1. End-to-end data flow

```
Admin upload / Zoho image sync
    → S3/R2 (original + .display.jpg + .thumb.jpg)
    → PostgreSQL (images_json, color_images_json, colors_json)
    → GET /api/v1/products (ProductDTO)
    → mapProduct() in storefront.ts
    → product.$slug.tsx
    → galleryForSelectedColor(product, color, size)
    → ProductGallery + VariantColorButtons (LazyImage)
```

Upload path (admin):

1. Browser normalizes file client-side (`product-image-normalize.ts` — gallery longest edge 1250px).
2. `POST /api/v1/admin/uploads` stores original + generates derivatives (`ImagePreviewSupport`, `S3StorageService`).
3. URLs are saved on the product row via `ProductColorImagesSupport` (per-print map) and/or `images_json`.

Storefront read path:

1. `useProducts` / PDP loader fetches product JSON.
2. `galleryForSelectedColor()` picks URLs for the active print.
3. `storefrontImageUrl(url, "display" | "thumb")` rewrites to derivative siblings.
4. `LazyImage` renders with appropriate `priority`; `prefetchStorefrontImage()` warms cache for neighbours and other prints.

---

2. Gallery resolution (`galleryForSelectedColor`)

File: `src/lib/product-gallery.ts` (mirrored at repo root `lib/product-gallery.ts` for Vitest paths).

Entry point:

```
galleryForSelectedColor(
  product: ProductGallerySource,
  selectedColor?: string | null,
  selectedSize?: string | null,
  selectedSku?: string | null,
): { image: string; gallery: string[] }
```

Returns `{ image: heroUrl, gallery: remainingUrls[] }`. Hero is always first; `gallery` is lifestyle shots only (no duplicate of hero).

Resolution order (simplified)

Step	Condition	Result
1	Variant row for size+colour+SKU has `imageUrl` / `imageUrls`	Variant photos; other sizes' fronts stripped from lifestyle list
2	Same-colour variants share one photo set (donor fallback)	Donor variant URLs
3	Variant exists but colour has variant photos elsewhere	Product hero, empty gallery (avoid wrong SKU shot)
4	`printsHaveDistinctGalleries(colors)`	**Only** selected print's URLs from `galleryUrlsForStorefrontColor()`
5	Shared or overlapping per-print galleries	`filterGalleryUrlsForPrint()` filename heuristics
6	Per-print product, selected print has no photos	`fallbackProductHero()` — product-level hero, **swatch URLs excluded**
7	Legacy single gallery	`product.image` + `product.gallery`

Important invariants

- **Catalog order preserved** — landscapes are not regrouped ahead of portraits (`600a6d5`).
- **Swatches ≠ gallery** — `swatchUrl` is for the colour picker only; `normalizeStorefrontColors()` removes swatch URLs from lifestyle lists when both exist.
- **Aspect-aware hero** — `galleryHeroFitSize()` scales to max 1250px longest edge without forcing square (`PRODUCT_GALLERY_HERO_MAX_PX`).

Helpers in `product-colors.ts`: `printsHaveDistinctGalleries`, `galleryUrlsForStorefrontColor`, `filterGalleryUrlsForPrint`, `swatchImageForColor`.

---

3. Image derivatives (`storefront-image`)

File: `src/lib/storefront-image.ts`

R2/S3 stores three tiers for each catalog photo:

`StorefrontImageSize`	Filename suffix	Width (longest edge)	Typical use
`thumb`	`.thumb.jpg`	240px	Side rail, swatches, blur LQIP
`display`	`.display.jpg`	800px	PDP hero (primary request)
`original`	(base path)	1250px	`srcset` high-DPI / fallback

API

```
storefrontImageUrl(url, "display" | "thumb" | "original")
storefrontImageSrcSet(url)  // "display 800w, original 1250w" for catalog HTTPS URLs
prefetchStorefrontImage(url, size?)  // deduped new Image() warm load
prefetchStorefrontImages(urls, size?, { idle?: boolean })
resetStorefrontImagePrefetch()  // test helper
```

Rewriting rules:

- Only `https://` catalog URLs are rewritten (`isRewritableCatalogUrl`).
- `data:`, `blob:`, and same-origin `/uploads/...` paths pass through unchanged.
- Existing `.thumb.jpg` / `.display.jpg` suffixes are stripped before re-applying the requested size.

Constants for `sizes` attribute:

- `STOREFRONT_HERO_SIZES` — PDP hero
- `STOREFRONT_THUMB_SIZES` — `"76px"` (side rail max)
- `STOREFRONT_SWATCH_SIZES` — colour picker

---

4. LazyImage component

File: `src/components/LazyImage.tsx`

Shared image primitive used across storefront (PDP, cards, cart, navbar, homepage). Wraps native `<img>` with loading policy and resilience.

Props

Prop	Type	Purpose
`src`	`string`	Primary URL (usually derivative)
`fallbackSrc`	`string?`	Original URL if derivative fails
`priority`	`high | eager | low | auto`	Controls `loading` and `fetchPriority`
`srcSet` / `sizes`	standard	Passed through unless on fallback

Behaviour

```
loading = priority === "high" || priority === "eager" ? "eager" : "lazy"
fetchPriority = priority === "high" ? "high" : priority === "low" ? "low" :
undefined
```

On mount, calls `loadImage(trimmed)` from `@/lib/image-cache` for same-origin URLs (in-memory LRU blob cache). Cross-origin CDN images rely on browser HTTP cache (`shouldBypassBlobCache` returns true for CDN).

On `error`, if `fallbackSrc` differs from `src`, swaps to fallback and clears `srcSet`.

Priority usage on PDP

Element	`priority`	`fetchPriority`	`src` tier
Hero	`high`	`high`	`.display.jpg` + `srcset`
Blur placeholder	`eager`	`low`	`.thumb.jpg`
Side thumb (first 6)	`eager`	`low`	`.thumb.jpg`
Side thumb (7+)	`low`	`low`	`.thumb.jpg`
Swatch (first 8)	`eager`	`low`	`.thumb.jpg`
Swatch (9+)	`low`	`low`	`.thumb.jpg`
“You may also love” cards	`low` (via `imagePriority`)	—	`.display.jpg`

`LazyImage` and `prefetchStorefrontImage()` are complementary: LazyImage controls **when** the DOM requests an image; prefetch warms the browser cache **ahead** of navigation (next/prev slide, hover on swatch).

---

5. ProductGallery UI

File: `src/components/ProductGallery.tsx`

Layout

- **Desktop:** side thumbnail rail (left) + hero (right), sticky on PDP (`lg:sticky lg:top-2`).
- **Mobile:** hero first (`order-1`), horizontal thumb scroller below (`order-2`).
- Cream frame (`bg-cream`), rounded corners, `object-contain` (no forced square crop).

Hero frame sizing

1. `useImageNaturalSize()` caches `naturalWidth` / `naturalHeight` per URL.
2. `galleryHeroFitSize()` computes CSS box from aspect ratio (max 1250px).
3. `lastFrameRef` keeps previous frame during image swap to avoid layout jump.
4. Padding-bottom spacer maintains aspect ratio before image paints.

Blur-up (LQIP)

While hero loads, an absolutely positioned blurred `.thumb.jpg` sits behind the hero (`blur-md scale-105`). Same URL family as side thumbs — tiny download, immediate paint.

Thumbnail rail

- `GALLERY_EAGER_THUMB_COUNT = 6` — first six thumbs use `loading="eager"`.
- Remaining thumbs use `loading="lazy"`.
- Rail `maxHeight` tracks hero frame height on desktop.

Prefetch (`useEffect`)

After hero natural dimensions are known:

1. **Immediate:** prefetch next + previous slide at `display` size.
2. **Idle (`requestIdleCallback`, 1.8s timeout):** prefetch rest of current gallery + `relatedImages` (other prints' heroes from PDP).

`relatedImages` is keyed via `relatedKey` (`join("\0")`) to avoid effect churn from default `[]` reference.

---

6. Colour swatches & prefetch on intent

File: `src/components/VariantPickers.tsx` — `VariantColorButtons`

Feature	Implementation

Swatch image	`swatchImageForColor(c)` → `storefrontImageUrl(..., "thumb")`
Hover / focus warmup	`onColorIntent(colorName)` callback
PDP handler	`prefetchColorGallery` in `product.$slug.tsx` prefetches that print's hero + `gallery[0]` at display size
Quick View	Same `onColorIntent` pattern in `QuickView.tsx`

Swatch images render at 48–56px; requesting 240px thumbs is intentional (sharp on retina, still far smaller than 800px display files).

---

7. PDP route wiring

File: `src/routes/product.$slug.tsx`

```
const colorGallery = useMemo(
  () => galleryForSelectedColor(product, color, size),
  [product, color, size],
);

const relatedHeroes = useMemo(() => {
  // One hero URL per other available print — fed to ProductGallery for
idle prefetch
}, [product, availableColors, size, colorGallery.image]);

<ProductGallery
  image={colorGallery.image}
  gallery={colorGallery.gallery}
  relatedImages={relatedHeroes}
  ...
/>

<VariantColorButtons
  onColorIntent={prefetchColorGallery}
  ...
/>
```

Colour/size state drives gallery recomputation; `useLayoutEffect` in `ProductGallery` resets active index to 0 when the image list changes.

---

8. Backend & admin

Area	Files	Notes

Per-print gallery storage	`ProductColorImagesSupport.java`	`color_images_json` map; `applyGalleryMapToColorsJson`
Product save	`ProductService.java`	Merges gallery map into `colors_json`
Zoho image sync	`ZohoItemImageSync.java`	Pull/push variant photos; lifestyle on colour gallery for simple products
Admin save stability	`ProductVariantSupport.java`, `a6f301f`	Print labels, swatches, galleries less likely to drop on save after Zoho pull
Derivative generation	`ImagePreviewSupport.java`, `S3StorageService.java`	`.thumb.jpg` / `.display.jpg` on upload
Preview API	`ImagePreviewService.java`, `InternalMediaPreviewResource.java`	Admin preview of derivatives
Client upload normalize	`product-image-normalize.ts`	Gallery export 1250px; swatch smaller
Admin UI	`ColorImagesEditor.tsx`	Per-print lifestyle upload; `ADMIN_IMAGE_SPECS.productGallery`

Admin image specs: `src/lib/admin-image-specs.ts` — documents that swatch uploads are for the colour picker, not the main gallery.

---

9. Performance strategy

Goals

1. Minimize **LCP** — hero visible before thumb strip completes.
2. Minimize **bytes on critical path** — display (800px) not original (1250px) for first paint.
3. Make **print switches** feel instant — prefetch on hover + idle warmup of sibling prints.
4. Avoid **priority inversion** — recommendations and deep thumbs must not compete with hero.

Sequence (typical PDP load)

```
1. HTML/JS hydrate
2. LazyImage hero: GET *.display.jpg (fetchpriority=high, eager)
3. LazyImage blur + first 6 thumbs: GET *.thumb.jpg (eager,
fetchpriority=low)
```

```
4. Hero onLoad → natural size → frame layout settles
5. prefetchStorefrontImage(next, prev) at display size
6. requestIdleCallback → prefetch remaining gallery + related print heroes
```

What we deliberately do *not* do

- Prefetch full 1250px originals for every thumb on mount (removed in `b1feaa2`).
- Regroup gallery by orientation (removed in `600a6d5`).
- Load all gallery `display` files before hero (`b1feaa2`).

---

10. Testing

Run focused suite:

```
npx vitest run \
  src/components/ProductGallery.test.tsx \
  src/components/VariantPickers.test.tsx \
  src/lib/storefront-image.test.ts \
  src/lib/product-gallery.test.ts \
  src/lib/product-colors.merge.test.ts
```

Test file	Covers
`ProductGallery.test.tsx`	Hero `src`/`srcset`/priority, cream frame, thumb eager/lazy split, prefetch spy, DOM order (hero before thumbs)
`VariantPickers.test.tsx`	Swatch `.thumb.jpg` URL, `onColorIntent` on pointer enter
`storefront-image.test.ts`	URL rewriting, `srcset`, prefetch dedup
`product-gallery.test.ts`	Per-print resolution, order, fallbacks, `galleryHeroFitSize`
`product-colors.merge.test.ts`	No cross-print gallery mixing when distinct galleries exist

Backend: `ProductColorImagesSupportTest`, `ZohoItemImageSyncTest` (gallery replace/combine scenarios).

---

11. Troubleshooting

Symptom	Likely cause	Check

Wrong print's photos on PDP	Shared gallery overlap / missing per-print map	`color_images_json`, `printsHaveDistinctGalleries`
Empty PDP, swatch still shows	Swatch-only print (by design)	Upload lifestyle gallery for that print
Hero slow, thumbs fast	Priority inversion or missing `.display.jpg` on CDN	Network tab: hero should be `*.display.jpg`, `fetchpriority=high`
404 on `.thumb.jpg`	Derivative not generated on upload	Re-upload or run preview/regen pipeline
Gallery order wrong	Old orientation sort (should be fixed)	Confirm `product-gallery.ts` has no sort-by-aspect
Colour switch laggy	Prefetch not firing	`onColorIntent` wired on PDP; hover before click
Fallback to huge original	Derivative missing; `fallbackSrc` used	S3 keys for `.display.jpg` sibling

Manual UAT checklist:

1. Product with 2+ prints, 3+ photos each.
2. Confirm thumb order matches admin upload order.
3. Switch print — gallery resets to index 0, new photos only.
4. Network: hero `.display.jpg` first; swatches `.thumb.jpg`.
5. Hover second print — display URL prefetched before click.

---

12. Future work

Not implemented; candidates if profiling shows need:

- `preconnect` / `dns-prefetch` for CDN host in `__root.tsx`
- WebP/AVIF derivatives (backend + `storefront-image` suffix map)
- Service worker / longer-lived cache for repeat visits
- RUM: LCP, CLS on PDP with real CDN latency
- Responsive `srcset` for thumb tier (currently single 240px file)

---

13. Code index

Topic	Path
Gallery resolution	`src/lib/product-gallery.ts`

Print / swatch helpers	`src/lib/product-colors.ts`
Derivative URLs + prefetch	`src/lib/storefront-image.ts`
Lazy load wrapper	`src/components/LazyImage.tsx`
In-memory image cache	`src/lib/image-cache/`
PDP gallery UI	`src/components/ProductGallery.tsx`
Colour swatches	`src/components/VariantPickers.tsx`
PDP page	`src/routes/product.$slug.tsx`
Product card (listing)	`src/components/ProductCard.tsx`
Client upload normalize	`src/lib/product-image-normalize.ts`
Storefront product map	`src/lib/storefront.ts`
Per-print DB support	`backend/.../ProductColorImagesSupport.java`
Derivative generation	`backend/.../ImagePreviewSupport.java`
Zoho images	`backend/.../ZohoItemImageSync.java`

Notable commits

Commit	Summary
`600a6d5`	Keep PDP gallery in catalog upload order
`a6f301f`	PDP framing; Zoho/admin media stability on save
`b1feaa2`	Hero-first loading; limit upfront prefetch
`028d1c9`	Size notes, image preview API, storefront image handling
2026-09-16	Thumb/swatch derivatives, idle prefetch, `onColorIntent`, `LazyImage` `eager` tier